cheesecracker

The API server of the Cheese App, built from an empty folder.

- Tests: unit, integration and system
- Checks: coverage, ruff and pre-commit
- Docker: a run image and a dev image

The tool uv installs Python and the packages of the project. The tool just runs each task by a short name. Everything except the Docker images runs on this machine with these two tools.

Unit tests

A unit test calls one function alone, with no server and no network.

Tool	Does
`uv add --group test pytest`	adds pytest to the group `test`
`just test`	runs `uv run pytest`

The API server

FastAPI turns Python functions into answers to HTTP requests. uvicorn runs the app.

Request	Answer
`/`	a welcome
`/health`	`{"status": "healthy"}`

`just serve` starts it in the background at `http://localhost:9000`, and it restarts on each change. `just stop` stops it.

The testing pyramid

Kind	Calls	Needs
Unit	one function	nothing
Integration	the app, through `TestClient`	no server
System	the running server, over HTTP	a server at port 9000

Unit tests are the base of the pyramid, many and fast. System tests are its top, few and slow.

Coverage

`just cov` runs every test. Then it lists each line of the code of the server, in the package `api`, that no test ran.

- The unit tests alone cover 11% of the package.
- All the tests cover 95%. No test asks for `/health`.

The formatter and the linter

ruff formats the code and checks it against rules. It does the work of two older tools, black, a formatter, and flake8, a linter.

Recipe	Runs	Fixes
`just fmt`	`ruff format`	spaces, blank lines, line breaks
`just lint`	`ruff check`	nothing alone. `just lint --fix` fixes some rules
`just check`	format, lint, tests	nothing: it only checks

Checks before every commit

pre-commit reads `.pre-commit-config.yaml`, and runs its checks before each `git commit`.

- pre-commit-hooks: whitespace, YAML, JSON, large files, private keys
- ruff-pre-commit: `ruff-check` and `ruff-format`

`just hook` installs the hook, once in each clone.

The run image

"It works on my machine." "Then we will ship your machine."

A Docker image holds a small Linux system with a program and all that it needs, so it runs the same on any computer.

Stage	Holds
`uv`	uv, from its own image
`build`	the dependencies of the server, from `uv.lock`, in `/opt/venv`
`run`	Python, `/opt/venv` and the code. It starts uvicorn

The run image has no uv, no tests and no ruff. It is 173 MB.

System tests

Real HTTP requests go to the server in the run image, the server as it runs for the users of the app.

Recipe	Does	Needs a server
`just test`, `just check`	the unit and integration tests	no
`just docker-serve`	starts the run image at port 9000	
`just test-system`	sends the requests	yes, and fails without one
`just docker-stop`	stops the server	

A server is a fixture of the commands that need it. Start it, use it, and stop it.

Two images from one lockfile

Image	Holds	Starts	Size
run	the server and its dependencies	uvicorn	173 MB
dev	the project, every group, uv, just and git	a shell	353 MB

`just docker-test` runs `just check` in the dev image, with this folder mounted at `/work`.

A new endpoint

`/add/` adds two numbers, and an integration test calls it.

1. `just check` on this machine
2. `just docker-test` in the dev image
3. `just docker-build run` builds the run image again, with the new code, and `just docker-serve` starts it
4. `just test-system` against the new run image, then `just docker-stop`